QC Trace.

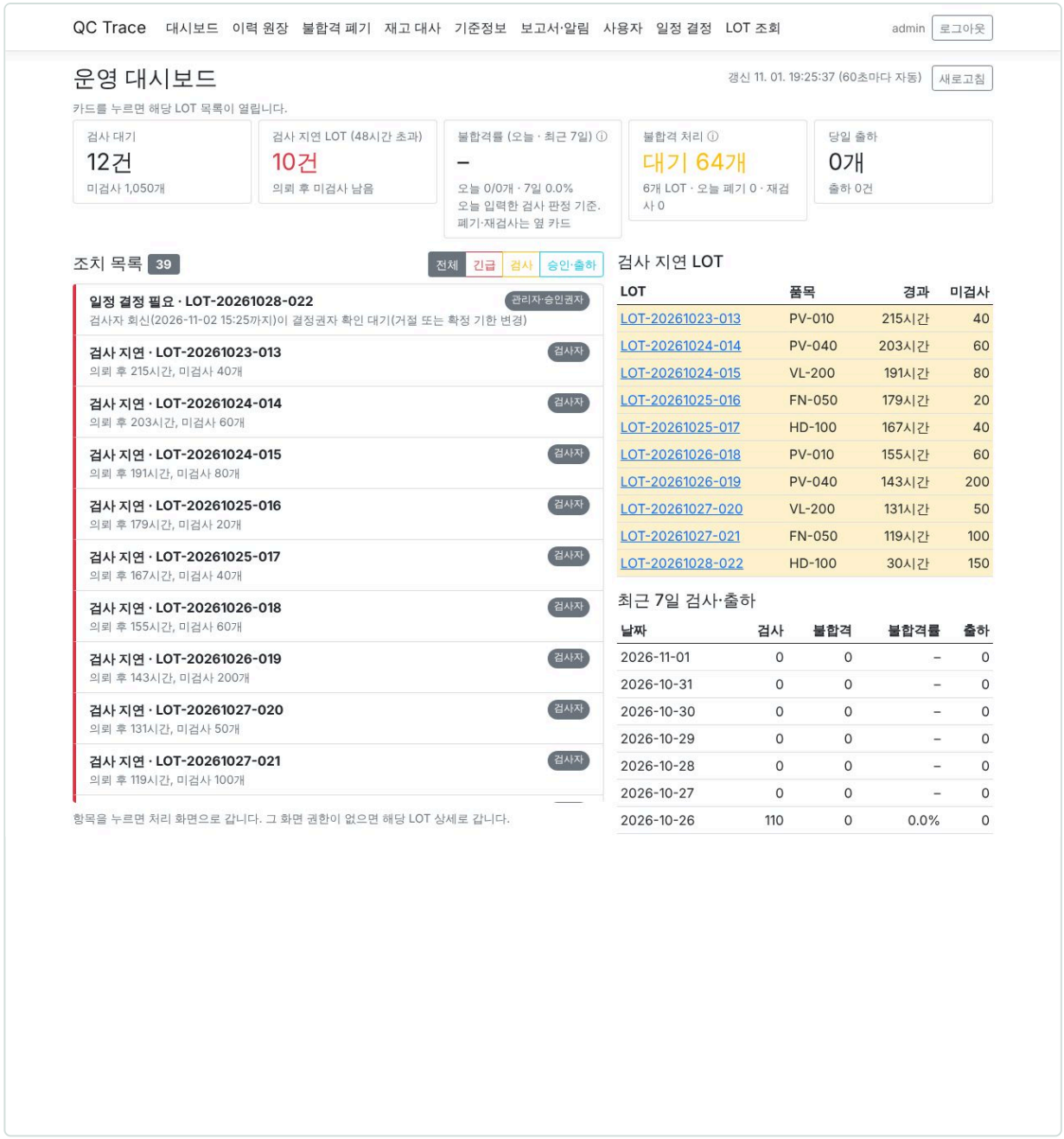
제조 LOT 검사·출하 이력 관리 시스템

.NET 8 · ASP.NET Core · SQL Server 2022 · T-SQL 저장 프로시저 · EF Core · JavaScript · Docker · xUnit · Playwright

전우석 · 1인 개발 · 2026.10.31 ~ 11.01 (2일)

기획 · 설계 · 업무 규칙 정의 · 검증

Claude Code 활용 구현



운영 대시보드 · 가상 데이터

검사를 통과한 수량만 출하되도록, 규칙을 DB에서 강제하는 사내 시스템

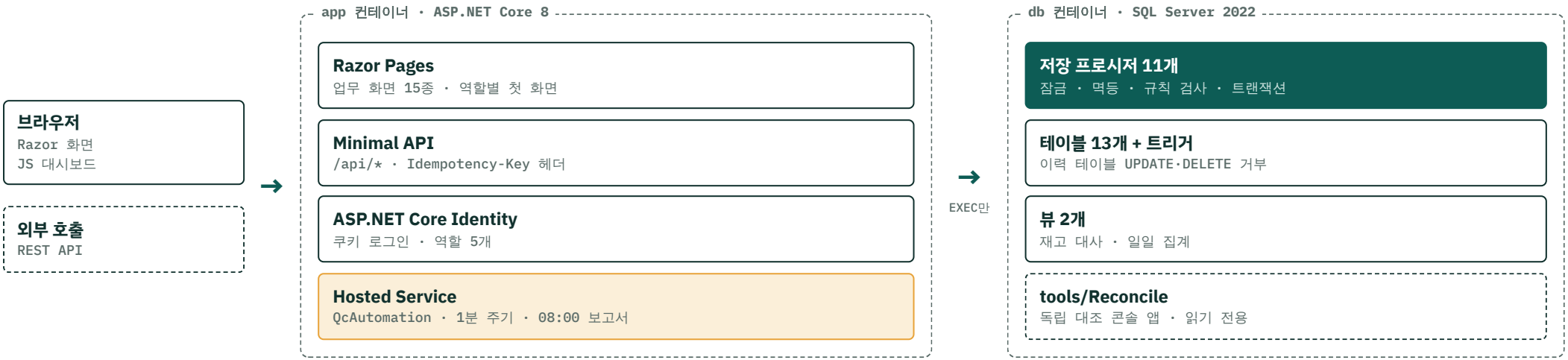
- **검사 → 승인 → 출하** 순서를 건너뛸 수 없습니다. 합격 수량만 출하 대상입니다.
- **독립 승인** 검사한 사람은 그 LOT의 출하를 승인할 수 없습니다.
- **불변 이력** 모든 수량 변경이 지울 수 없는 원장에 작업자·시각·사유와 함께 남습니다.

13 업무 테이블	**11** 저장 프로시저	**15** 화면
5 사용자 역할	**11** 업무 규칙 (R1~R11)	**237** 자동화 테스트



LOT 상세: 수량 · 검사 기록 · 승인·출하 · 전체 이력

컨테이너 두 개, 수량 변경은 저장 프로시저 한 길로



`volume · mssql-data` `volume · reports (엑셀)` `volume · dp-keys (쿠키 암호화 키)`

로컬 검증 `docker compose up` → `dotnet test · 규칙 81 + 통합 156` → `Playwright E2E 7` → `독립 대조 불일치 0`

- **실행 연결** 앱은 최소 권한 계정 `qctrace_app`으로 접속. 수량·이력 테이블 쓰기 DENY, 저장 프로시저 EXECUTE만 허용.
- **관리자 연결** `sa`는 기동 시 마이그레이션·계정 생성·시드에만 사용.

사내 시스템에서 흔한 조합을, 계층마다 이유를 갖고

계층	선택	이유
런타임	**.NET 8 LTS · C#**	장기 지원 버전, 사내 시스템 유지보수에 적합
웹	**Razor Pages + Minimal API**	화면과 API를 한 프로젝트에, 과한 구조 없이
DB	**SQL Server 2022**	트랜잭션·행 잠금·트리거·권한을 DB 수준에서 사용
DB 접근	**EF Core + 저장 프로시저**	조회·마이그레이션은 EF Core, 수량 변경은 SP 안에서 잠금과 함께
인증	**ASP.NET Core Identity**	기본 제공 쿠키 세션, 역할 기반 정책
프론트	**Razor + 순수 JavaScript**	대시보드는 프레임워크 없이 fetch로, 의존성 최소화
자동화	**Hosted Service + ClosedXML**	별도 스케줄러 없이 앱 안에서 엑셀 보고서 생성
테스트	**xUnit · Testcontainers · Playwright**	가짜 DB가 아닌 실제 SQL Server로 검증, 브라우저 시나리오까지
배포	**Docker Compose**	앱 + DB를 명령 한 번으로 재현

04 데이터 모델

현재 상태와 지울 수 없는 이력을 분리했습니다

기준정보

Items
ItemCode · Name · Spec

InspectionItems
ItemId · Lower · Upper · Unit

DefectCodes
Code · Name · IsActive

Users · Roles
ASP.NET Core Identity

현재 상태 · 빠른 조회용

Lots
LotNo · ItemId · ProducedQty
Status · DueAt · RowVersion

LotQuantities
Uninspected · Passed · Failed
Held · Scrapped · Reserved · Shipped

운영

QcRequests
RequestKey (UNIQUE) · BodyHash
ResponseJson

DelayAlerts
LotId (UNIQUE) · DetectedAt

추가 전용 이력 · UPDATE·DELETE 트리거 거부

Ledger
LotId · DeltaField · Delta · ActorId
Reason · SourceTable · SourceId

Inspections
Qty · Verdict · MeasuredJson · Reason
CorrectsInspectionId (정정은 새 행)

ShipmentApprovals · Shipments
ApproverId · ShipperId · Qty

ScheduleEvents · RoleChanges
일정 협의 · 역할 변경 이력

LotQuantities = Σ Ledger · 현재 수량은 언제든 원장 합으로 다시 계산할 수 있어야 합니다. 원장 행은 근거 행(SourceTable · SourceId)을 가리킵니다.

요청 하나가 저장 프로시저 안에서 처리되는 순서

1 요청
화면은 폼마다 새 요청 키, API는 `Idempotency-Key` 헤더

2 입력 검사
수량·판정·사유를 트랜잭션 전에 확인

3 요청 키 조회
본문 SHA-256 비교. 같으면 저장된 응답, 다르면 `50409`

4 LOT 잠금
`UPDLOCK, ROWLOCK` 같은 LOT만 대기

5 규칙·기록
R1~R11 검사 → 기록 행 + 원장 행 + 수량 갱신

6 응답 저장
응답 JSON을 요청 키와 함께 저장 후 COMMIT

`BEGIN TRANSACTION ———— COMMIT · 실패 시 XACT_ABORT로 전체 롤백`

SQL 오류	API 응답	의미
50403	**403**	역할 없음
50409	**409**	같은 키, 다른 내용 (R6)
52004	**422**	사유 누락 (R4)

- THROW 번호를 HTTP 상태로 한곳에서 변환해, 화면과 API가 같은 규칙 메시지를 씁니다.
- 재전송이면 **`Idempotent-Replayed`** 헤더로 알리고, 키가 없으면 400입니다.

멱등 처리, 행 잠금, 원자적 기록을 한 프로시저에

```
SET XACT_ABORT ON;
DECLARE @BodyHash char(64) = CONVERT(char(64), HASHBYTES('SHA2_256',
  CONCAT_WS(N'|', @Operation, @LotId, @InspectorId, @Qty, @Verdict, ...)), 2); ①
BEGIN TRANSACTION;
SELECT @SavedHash = BodyHash, @SavedResponse = ResponseJson
FROM dbo.QcRequests WITH (UPDLOCK, HOLDLOCK) WHERE RequestKey = @RequestKey; ②
SELECT @LotStatus = Status FROM dbo.Lots WITH (UPDLOCK, ROWLOCK) WHERE Id = @LotId; ③
-- … R11 규격 검사, R2·R9 잔량·초과분 계산 생략
INSERT INTO dbo.Ledger (LotId, Kind, DeltaField, Delta, ActorId, Reason, SourceTable, SourceId, CreatedAt) ...;
UPDATE q SET Uninspected = ..., Passed = ... FROM dbo.LotQuantities q WHERE q.LotId = @LotId;
INSERT INTO dbo.QcRequests (RequestKey, Operation, BodyHash, ResponseJson, CreatedAt) ...;
COMMIT TRANSACTION; ④
```

① 본문 해시
같은 키에 다른 내용이 오면 거절합니다.

② HOLDLOCK
같은 키 동시 요청은 첫 요청의 커밋을 기다립니다.

③ UPDLOCK
같은 LOT만 직렬화, 다른 LOT은 병렬로 진행합니다.

④ 원자성
원장·수량·응답이 함께 커밋되거나 함께 롤백됩니다.

화면을 우회해도 막히도록, 세 겹으로

1. 웹 계층 ASP.NET Core Identity	역할별 접근 정책. 역할 없는 사용자는 업무 화면 접근 불가. 본인 검사 LOT은 승인 버튼 비활성화.	HTTP 403
2. 저장 프로시저 업무 규칙의 최종 판단	작업자 역할 재확인, 검사자 본인 승인 거절(R5), 폐기는 관리자만, 사유 필수(R4). 화면 검증과 무관하게 항상 실행.	THROW 50403 · 52004
3. DB 권한·트리거 최소 권한 계정	앱 계정은 수량·이력·역할 테이블 직접 쓰기 DENY. 이력 테이블은 트리거가 UPDATE·DELETE 거부. 역할 변경은 usp_SetUserRoles로만, RoleChanges에 기록.	SQL 오류 229

통합 테스트로 확인: 앱 계정 직접 수정 시도 6종 거부 · 화면에 숨겨 보낸 폐기 요청 거절 · 역할 없는 사용자 403 · 입력값의 스크립트가 화면에서 실행되지 않음

동시에 들어와도 수량이 틀리지 않고, 틀리면 찾아냅니다

LOT 단위 직렬화



격리 수준은 기본 READ COMMITTED · LOT 행에 UPDLOCK·ROWLOCK · 요청 키 조회에 UPDLOCK·HOLDLOCK

동시 20건 테스트	결과
승인 각 10개 · 출하 가능 100	**10 성공 · 10 거절**
출하 각 10개 · 승인 50	**5 성공 · 15 거절**
같은 요청 키 20번	**1 반영 · 19 재전송 응답**
검사·승인·출하 섞어서	**교착 0 · 등식 유지**

두 겹의 대사

vw_LotReconciliation · DB 안
현재 수량 = 원장 합
생산 = 미검사 + 합격 + 불합격 + 보류 + 폐기

tools/Reconcile · 앱 밖
앱 코드·EF 모델·뷰를 쓰지 않고 원본 행만 읽어 재계산
원장 행마다 근거 행의 수량·방향까지 확인



독립 대조 LOT 33건 불일치 0 · SP를 거치지 않은 조작 6종 검출

스케줄러 없이 앱 안에서 돌고, 프레임워크 없이 그립니다

Hosted Service · QcAutomation

매 1분
지연 LOT 탐지
기한 또는 의뢰 후 48시간 초과

기록
DelayAlerts
LOT당 1회, 수정 불가

08:00
일일 보고서
집계 뷰 → ClosedXML → .xlsx

- 파일이 있으면 다시 만들지 않고, 그 시각에 꺼져 있었으면 다음 기동 때 만듭니다.



순수 JavaScript 대시보드

wwwroot
HTML · JS
프레임워크 없음

fetch
/api/dashboard
관리자 전용 · 원본 테이블 집계

60초
자동 갱신
카드 → LOT 목록 이동

- 조치 목록이 일정 결정·검사 지연 같은 할 일을 담당 역할과 함께 보여 줍니다.



LOT 100개: 등록부터 출하, 원장 확인까지



① 생산 담당
등록 → 검사 의뢰



② 검사자
60 합격, 잔량 40 표시



③ 승인권자
본인 검사 LOT 승인 차단



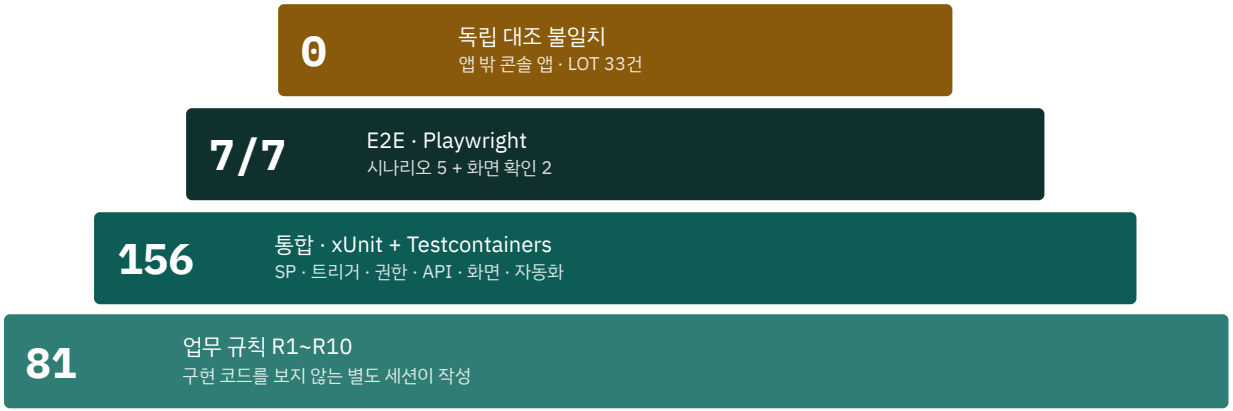
④ 출하 담당
승인 95에 100 → 거절



⑤ 관리자
원장 7줄로 전 과정 확인

그 밖의 화면: 불합격 처리·관리자 폐기 · 불량 유형 관리 · 검사 일정 협의 · 재고 대사 · 보고서·알림 · 기준정보 CSV · 사용자·역할 관리

실제 SQL Server 위에서 네 층으로 검증했습니다



- **실제 DB** Testcontainers로 SQL Server 컨테이너를 띄워 트리거·잠금·권한까지 검증
- **규칙 추적** R1~R11 각각을 대응 테스트와 문서로 연결
- **E2E가 찾은 결함** 전량 재검사 후 LOT이 목록에서 사라지는 상태 전이 오류 → 수정·회귀 테스트

> 2026-11-01 로컬 Docker 실행 결과 · 성능 측정은 하지 않음

로컬 검증 `docker compose up` → `dotnet test · 규칙 81 + 통합 156` → `Playwright E2E 7` → `독립 대조 불일치 0`

AI가 만들고, 다른 AI가 확인하고, 판단은 사람이



발견 경로	문제	조치
검증 세션	서버가 규격 밖 측정값의 합격을 받음	R11 신설, 통합 테스트 12건
E2E	전량 재검사 후 LOT이 검사 목록에서 사라짐	상태 SP 수정, 회귀 테스트
직접 사용	불합격이 남았는데 "출하 완료"로 표시	상태 SP 수정, 기존 데이터 보정

작업 단위 10개 + 사용 피드백 3회, 단위마다 테스트 통과 후 커밋 · 통합 테스트 31 → 156 · 코드 작성은 AI, 요구사항·규칙·검증 기준·업무 결정은 본인

하지 않은 것과 확인할 수 있는 곳

범위 밖 · 한계

- 성능 측정 없음 (수치를 적지 않음)
- 계측 장비·바코드·ERP·MES 연계 없음
- DB 관리자 권한으로 모든 행을 맞춰 조작하면 대조로 못 잡음
- 가상 데이터 기반 데모, 실운영 실적 아님

자료

GitHub 저장소 ↗ | 기획서 ↗ | 규칙·테스트 추적표 ↗ | 저장 프로시저 해설 ↗ | 개발 기록 ↗

> 실행: docker compose up → http://localhost:8080 · 역할별 데모 계정
> 화면은 커밋 88e6bbb · 가상 데이터 · 비밀번호·접속 정보 미포함